

CIS 6270

Module 4: Discrete Diffusion

Lecture 4.1: Masked Diffusion and MDLM

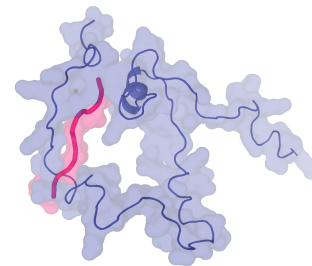
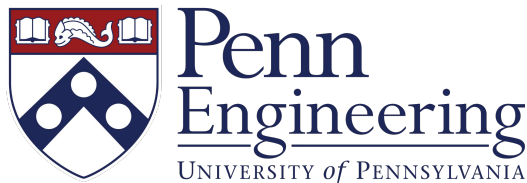
Tue, Sep 29, 2026

Pranam Chatterjee, PhD

Assistant Professor of Bioengineering

Assistant Professor of Computer and Information Science

University of Pennsylvania



Last time, we learned to reverse continuous noise

Last time, we learned to reverse continuous noise

In Lecture 3, corruption moved a continuous state toward noise. A learned reverse process brought it back toward the data distribution.

Last time, we learned to reverse continuous noise

In Lecture 3, corruption moved a continuous state toward noise. A learned reverse process brought it back toward the data distribution.

For text and other sequences, we can instead keep every state in a finite vocabulary and learn which tokens to reveal or replace.

Next-token prediction gives us a strong starting point

Next-token prediction gives us a strong starting point

GPT and Claude have shown how much language, reasoning, and code generation can grow from autoregressive prediction at scale.

Next-token prediction gives us a strong starting point

GPT and Claude have shown how much language, reasoning, and code generation can grow from autoregressive prediction at scale.

A completed prefix gives each new prediction useful context.

Next-token prediction gives us a strong starting point

GPT and Claude have shown how much language, reasoning, and code generation can grow from autoregressive prediction at scale.

A completed prefix gives each new prediction useful context. The natural next question is what changes when some useful context is already available on both sides.

A missing token may depend on what follows it

A missing token may depend on what follows it

The [MASK] was too hot to drink.

A missing token may depend on what follows it

The [MASK] was too hot to drink.

The words on either side help us predict the missing token.

A missing token may depend on what follows it

The [MASK] was too hot to drink.

The words on either side help us predict the missing token. The same idea supports text infilling and completing a partially observed sequence.

A missing token may depend on what follows it

The [MASK] was too hot to drink.

The words on either side help us predict the missing token. The same idea supports text infilling and completing a partially observed sequence.

If several positions are missing, a bidirectional model can use all currently visible tokens to predict them together.

How do these predictions become a generative model?

How do these predictions become a generative model?

Training on different missing subsets gives us many prediction tasks, from a nearly complete sequence to an entirely masked one.

How do these predictions become a generative model?

Training on different missing subsets gives us many prediction tasks, from a nearly complete sequence to an entirely masked one.

We need a corruption process, a loss, and valid reverse transitions that connect these predictions.

How do these predictions become a generative model?

Training on different missing subsets gives us many prediction tasks, from a nearly complete sequence to an entirely masked one.

We need a corruption process, a loss, and valid reverse transitions that connect these predictions. Masked diffusion gives us a concrete way to derive all three.

We will use four letters to work through the calculations

We will use four letters to work through the calculations

We will study models for language, peptides, DNA, and other discrete data.

We will use four letters to work through the calculations

We will study models for language, peptides, DNA, and other discrete data. For worked calculations, we will use A, C, G, and T so that we can compute each probability by hand.

We will use four letters to work through the calculations

We will study models for language, peptides, DNA, and other discrete data. For worked calculations, we will use A, C, G, and T so that we can compute each probability by hand.

We will also use short DNA sequences in the code, after introducing each method in its published application setting.

Even four letters give us many possible sequences

Even four letters give us many possible sequences

$$\mathcal{A} = \{A, C, G, T\}, \quad |\mathcal{A}^L| = 4^L, \quad 4^{10} = 1,048,576$$

Even four letters give us many possible sequences

$$\mathcal{A} = \{A, C, G, T\}, \quad |\mathcal{A}^L| = 4^L, \quad 4^{10} = 1,048,576$$

The DNA alphabet $\mathcal{A} = \{A, C, G, T\}$ gives sequence-space size $|\mathcal{A}^L| = 4^L$ for length L , so at $L = 10$ there are $4^{10} = 1,048,576$ sequences.

Even four letters give us many possible sequences

$$\mathcal{A} = \{A, C, G, T\}, \quad |\mathcal{A}^L| = 4^L, \quad 4^{10} = 1,048,576$$

The DNA alphabet $\mathcal{A} = \{A, C, G, T\}$ gives sequence-space size $|\mathcal{A}^L| = 4^L$ for length L , so at $L = 10$ there are $4^{10} = 1,048,576$ sequences.

L is the number of DNA positions. Each position has four possible letters.

Even four letters give us many possible sequences

$$\mathcal{A} = \{A, C, G, T\}, \quad |\mathcal{A}^L| = 4^L, \quad 4^{10} = 1,048,576$$

The DNA alphabet $\mathcal{A} = \{A, C, G, T\}$ gives sequence-space size $|\mathcal{A}^L| = 4^L$ for length L , so at $L = 10$ there are $4^{10} = 1,048,576$ sequences.

L is the number of DNA positions. Each position has four possible letters.

A generative model concentrates probability on useful sequences without listing the entire search space.

The function can depend on several positions together

The function can depend on several positions together

If the desired sequences are AA, CC, GG, and TT, the second base must agree with the first.

The function can depend on several positions together

If the desired sequences are AA, CC, GG, and TT, the second base must agree with the first.

A model that independently samples two uniform bases generates a matching pair only one-quarter of the time. We need to learn these dependencies.

We can train by hiding part of a known sequence

We can train by hiding part of a known sequence

We use each clean sequence as its own training target.

We can train by hiding part of a known sequence

We use each clean sequence as its own training target. We corrupt it, ask a network to recover the missing information, and repeat at many noise levels.

We can train by hiding part of a known sequence

We use each clean sequence as its own training target. We corrupt it, ask a network to recover the missing information, and repeat at many noise levels.

At generation time, the network receives noise and repeatedly predicts how to make the sequence more consistent with the training data.

With a mask, we know which positions need prediction

With a mask, we know which positions need prediction

In AmGT, every visible letter is trustworthy. The network only needs to infer the missing second base.

With a mask, we know which positions need prediction

In AmGT, every visible letter is trustworthy. The network only needs to infer the missing second base.

With masking, we can work through a complete derivation using a simple posterior and familiar token cross-entropies.

We can also corrupt a sequence by replacing its tokens

We can also corrupt a sequence by replacing its tokens

In AGGT, the second G might be an original letter or a corruption of C. The network must infer which letters need revision.

We can also corrupt a sequence by replacing its tokens

In AGGT, the second G might be an original letter or a corruption of C. The network must infer which letters need revision.

Uniform replacement permits repeated editing during generation.

We can also corrupt a sequence by replacing its tokens

In AGGT, the second G might be an original letter or a corruption of C. The network must infer which letters need revision.

Uniform replacement permits repeated editing during generation. Its reverse posterior and likelihood loss differ from the masked case because no absorbing mask identifies the corrupted positions.

Both choices let us generate sequences directly

Both choices let us generate sequences directly

We previously generated real-valued coordinates and continuous embeddings. Now we will generate discrete sequences, including language tokens, peptide residues, and DNA bases.

Both choices let us generate sequences directly

We previously generated real-valued coordinates and continuous embeddings. Now we will generate discrete sequences, including language tokens, peptide residues, and DNA bases.

We will construct random token changes, learn how to reverse them, and control the properties of the generated sequences.

First, specify the alphabet and sequence length

First, specify the alphabet and sequence length

$$x = (x^1, \dots, x^L) \in \mathcal{A}^L, \quad |\mathcal{A}^L| = K^L$$

First, specify the alphabet and sequence length

$$x = (x^1, \dots, x^L) \in \mathcal{A}^L, \quad |\mathcal{A}^L| = K^L$$

The sequence x contains L tokens, x^1 through x^L , each drawn from an alphabet A with K symbols, so the sequence space $\mathcal{A}^L$ contains K^L possible sequences.

First, specify the alphabet and sequence length

$$x = (x^1, \dots, x^L) \in \mathcal{A}^L, \quad |\mathcal{A}^L| = K^L$$

The sequence x contains L tokens, x^1 through x^L , each drawn from an alphabet A with K symbols, so the sequence space $\mathcal{A}^L$ contains K^L possible sequences.

A is the alphabet; x^i is the token at position i ; L is sequence length; K is alphabet size.

First, specify the alphabet and sequence length

$$x = (x^1, \dots, x^L) \in \mathcal{A}^L, \quad |\mathcal{A}^L| = K^L$$

The sequence x contains L tokens, x^1 through x^L , each drawn from an alphabet A with K symbols, so the sequence space $\mathcal{A}^L$ contains K^L possible sequences.

A is the alphabet; x^i is the token at position i ; L is sequence length; K is alphabet size.

A 12-residue peptide has 20^{12} possible canonical sequences. Enumerating their probabilities is impractical.

How can a discrete sequence change through continuous time?

How can a discrete sequence change through continuous time?

$$\gamma : [0, 1] \rightarrow \mathcal{A}^L \text{ continuous} \implies \gamma(t) = \gamma(0)$$

How can a discrete sequence change through continuous time?

$$\gamma : [0, 1] \rightarrow \mathcal{A}^L \text{ continuous} \implies \gamma(t) = \gamma(0)$$

If the sequence-valued path γ is continuous on the time interval from 0 to 1, then the sequence at every time t , $\gamma(t)$, equals its initial sequence $\gamma(0)$.

How can a discrete sequence change through continuous time?

$$\gamma : [0, 1] \rightarrow \mathcal{A}^L \text{ continuous} \implies \gamma(t) = \gamma(0)$$

If the sequence-valued path γ is continuous on the time interval from 0 to 1, then the sequence at every time t , $\gamma(t)$, equals its initial sequence $\gamma(0)$.

$\gamma(t)$ is a sequence-valued path; the finite state space has the discrete topology.

How can a discrete sequence change through continuous time?

$$\gamma : [0, 1] \rightarrow \mathcal{A}^L \text{ continuous} \implies \gamma(t) = \gamma(0)$$

If the sequence-valued path γ is continuous on the time interval from 0 to 1, then the sequence at every time t , $\gamma(t)$, equals its initial sequence $\gamma(0)$.

$\gamma(t)$ is a sequence-valued path; the finite state space has the discrete topology.

A continuous image of a connected set is connected, and the only connected subsets of a discrete space are singletons.

We must specify when a token changes and what replaces it

We must specify when a token changes and what replaces it

We assign a probability to each possible token replacement.

We must specify when a token changes and what replaces it

We assign a probability to each possible token replacement. Across repeated samples, the sequence probabilities can change smoothly while each sampled sequence changes by jumps.

We must specify when a token changes and what replaces it

We assign a probability to each possible token replacement. Across repeated samples, the sequence probabilities can change smoothly while each sampled sequence changes by jumps.

Later, we will evolve continuous probability vectors on the simplex and decode their endpoints into tokens.

Each sampled sequence will change through jumps

Each sampled sequence will change through jumps

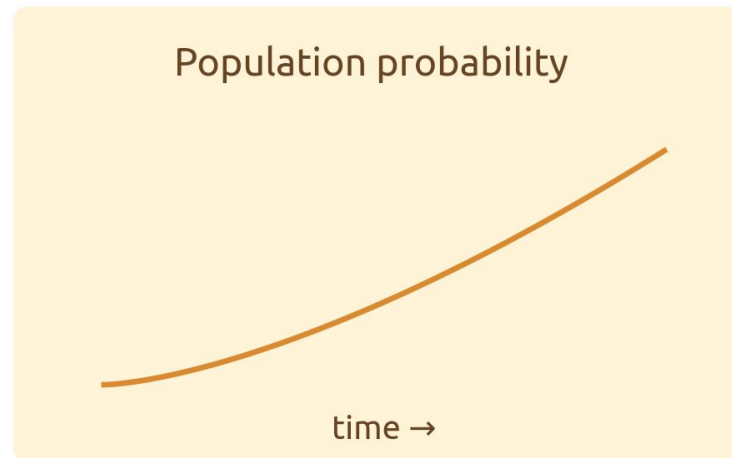
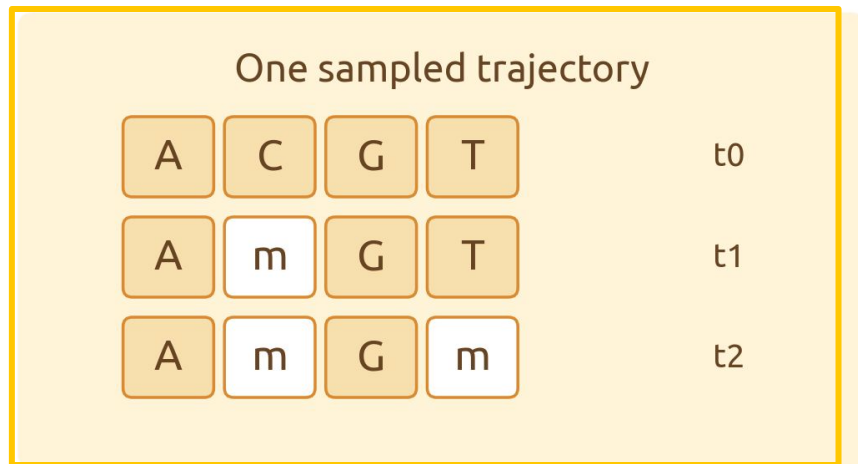
One sampled trajectory

A	C	G	T	t0
A	m	G	T	t1
A	m	G	m	t2

Population probability

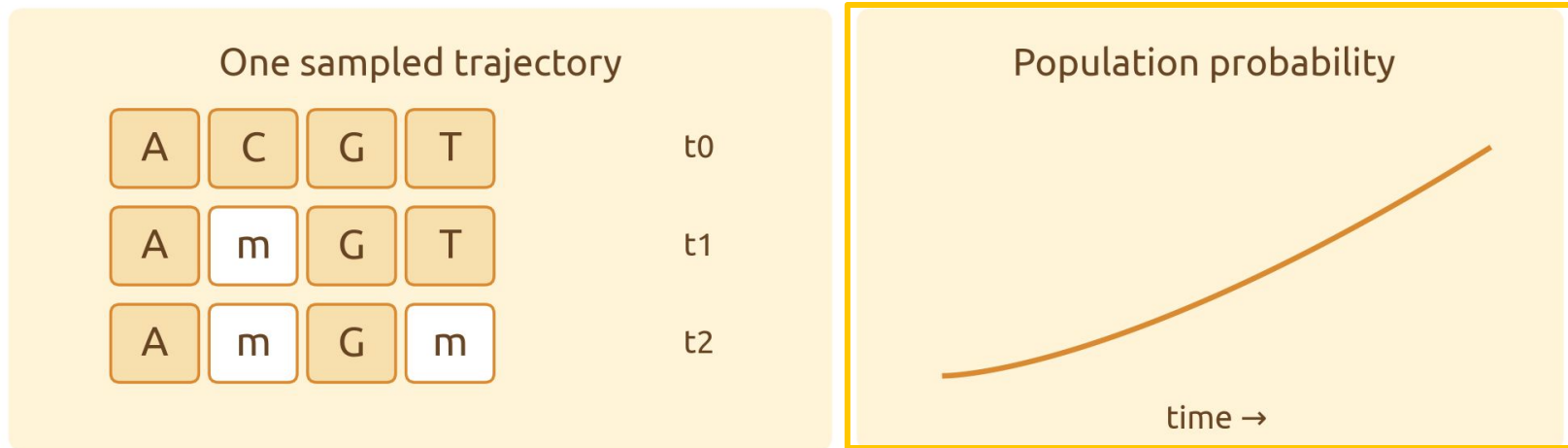


Each sampled sequence will change through jumps



The sampled sequence remains fixed between jump times.

Each sampled sequence will change through jumps



The sampled sequence remains fixed between jump times.

Across many independent samples, the frequency of each sequence changes smoothly.

We will begin by reversing a corruption process

We will begin by reversing a corruption process

We will erase tokens with a mask, replace tokens with uniform noise, and generate blocks conditioned on an existing prefix.

We will begin by reversing a corruption process

We will erase tokens with a mask, replace tokens with uniform noise, and generate blocks conditioned on an existing prefix.

At each reverse step, we will predict clean tokens using the entire noisy sequence.

MDLM does this by masking and reconstructing language

MDLM does this by masking and reconstructing language



NeurIPS 2024

Simple and Effective Masked Diffusion Language Models

Subham Sekhar Sahoo
Cornell Tech, NYC, USA.
ssahoo@cs.cornell.edu

Marianne Arriola
Cornell Tech, NYC, USA.
ma2238@cornell.edu

Yair Schiff
Cornell Tech, NYC, USA.
yzs2@cornell.edu

MDLM does this by masking and reconstructing language



NeurIPS 2024

Simple and Effective Masked Diffusion Language Models

Subham Sekhar Sahoo
Cornell Tech, NYC, USA.
ssahoo@cs.cornell.edu

Marianne Arriola
Cornell Tech, NYC, USA.
ma2238@cornell.edu

Yair Schiff
Cornell Tech, NYC, USA.
yzs2@cornell.edu

Sahoo et al. introduce MDLM for language modeling. We generate token sequences by learning to reverse progressive masking.

MDLM does this by masking and reconstructing language



NeurIPS 2024

Simple and Effective Masked Diffusion Language Models

Subham Sekhar Sahoo
Cornell Tech, NYC, USA.
ssahoo@cs.cornell.edu

Marianne Arriola
Cornell Tech, NYC, USA.
ma2238@cornell.edu

Yair Schiff
Cornell Tech, NYC, USA.
yzs2@cornell.edu

Sahoo et al. introduce MDLM for language modeling. We generate token sequences by learning to reverse progressive masking.

We will derive its weighted masked-token cross-entropy, then calculate and implement the loss with a four-letter DNA example.

Repeated masked-token prediction can generate a sequence

Repeated masked-token prediction can generate a sequence

As the masking rate varies across training examples, we can derive its loss weight from the diffusion objective.

Repeated masked-token prediction can generate a sequence

As the masking rate varies across training examples, we can derive its loss weight from the diffusion objective.

The same predictor can then define a sequence of valid reverse transitions from an all-mask sequence to clean data.

First, follow what happens to one clean token

First, follow what happens to one clean token

Introduce a special [MASK] symbol, denoted m , outside the clean alphabet.

First, follow what happens to one clean token

Introduce a special [MASK] symbol, denoted m , outside the clean alphabet.

During corruption, a clean token either remains visible or becomes m . Once masked, it stays masked.

We keep it visible with probability α_t

We keep it visible with probability α_t

Choose whether the original base survives.

$$B_t \sim \text{Bernoulli}(\alpha_t)$$

We keep it visible with probability α_t

Choose whether the original base survives.

$$B_t \sim \text{Bernoulli}(\alpha_t)$$

The survival draw B_t is Bernoulli with survival probability α_t , so it equals one with probability α_t and zero with probability one minus α_t .

We keep it visible with probability α_t

Choose whether the original base survives.

$$B_t \sim \text{Bernoulli}(\alpha_t)$$

Write the two possible outcomes.

$$Z_t = \begin{cases} a & B_t = 1 \\ m & B_t = 0 \end{cases}$$

We keep it visible with probability α_t

Choose whether the original base survives.

$$B_t \sim \text{Bernoulli}(\alpha_t)$$

Write the two possible outcomes.

$$Z_t = \begin{cases} a & B_t = 1 \\ m & B_t = 0 \end{cases}$$

The noisy token Z_t equals clean token a when survival draw B_t is one, and equals mask m when B_t is zero.

We keep it visible with probability α_t

Choose whether the original base survives.

$$B_t \sim \text{Bernoulli}(\alpha_t)$$

Write the two possible outcomes.

$$Z_t = \begin{cases} a & B_t = 1 \\ m & B_t = 0 \end{cases}$$

Add the probabilities of the two events.

$$q_t(z \mid a) = \alpha_t \mathbf{1}\{z = a\} + (1 - \alpha_t) \mathbf{1}\{z = m\}$$

We keep it visible with probability α_t

Choose whether the original base survives.

$$B_t \sim \text{Bernoulli}(\alpha_t)$$

Write the two possible outcomes.

$$Z_t = \begin{cases} a & B_t = 1 \\ m & B_t = 0 \end{cases}$$

Add the probabilities of the two events.

$$q_t(z \mid a) = \alpha_t \mathbf{1}\{z = a\} + (1 - \alpha_t) \mathbf{1}\{z = m\}$$

The noisy-token probability $q_t(z \mid a)$, given clean token a , equals survival probability α_t times the indicator that z equals a , plus masking probability one minus α_t times the indicator that z equals mask m .

The schedule determines how often the clean token survives

Choose whether the original base survives.

$$B_t \sim \text{Bernoulli}(\alpha_t)$$

Write the two possible outcomes.

$$Z_t = \begin{cases} a & B_t = 1 \\ m & B_t = 0 \end{cases}$$

Add the probabilities of the two events.

$$q_t(z \mid a) = \alpha_t \mathbf{1}\{z = a\} + (1 - \alpha_t) \mathbf{1}\{z = m\}$$

The noisy-token probability $q_t(z \mid a)$, given clean token a , equals survival probability α_t times the indicator that z equals a , plus masking probability one minus α_t times the indicator that z equals mask m .

Masking removes identity; it never changes one visible DNA base into another.

We apply this corruption independently at each position

We apply this corruption independently at each position

$$q_t(z \mid x) = \prod_{i=1}^L q_t(z^i \mid x^i)$$

We apply this corruption independently at each position

$$q_t(z \mid x) = \prod_{i=1}^L q_t(z^i \mid x^i)$$

The probability of the entire corrupted sequence z given the clean sequence x , $q_t(z \mid x)$, equals the product over positions $i = 1$ through L of the probability of noisy token z^i given clean token x^i , $q_t(z^i \mid x^i)$.

We apply this corruption independently at each position

$$q_t(z \mid x) = \prod_{i=1}^L q_t(z^i \mid x^i)$$

The probability of the entire corrupted sequence z given the clean sequence x , $q_t(z \mid x)$, equals the product over positions $i = 1$ through L of the probability of noisy token z^i given clean token x^i , $q_t(z^i \mid x^i)$.

x is the clean sequence; z is the noisy sequence; i indexes positions.

We apply this corruption independently at each position

$$q_t(z \mid x) = \prod_{i=1}^L q_t(z^i \mid x^i)$$

The probability of the entire corrupted sequence z given the clean sequence x , $q_t(z \mid x)$, equals the product over positions $i = 1$ through L of the probability of noisy token z^i given clean token x^i , $q_t(z^i \mid x^i)$.

x is the clean sequence; z is the noisy sequence; i indexes positions.

The corruptions are independent conditional on x .

We apply this corruption independently at each position

$$q_t(z \mid x) = \prod_{i=1}^L q_t(z^i \mid x^i)$$

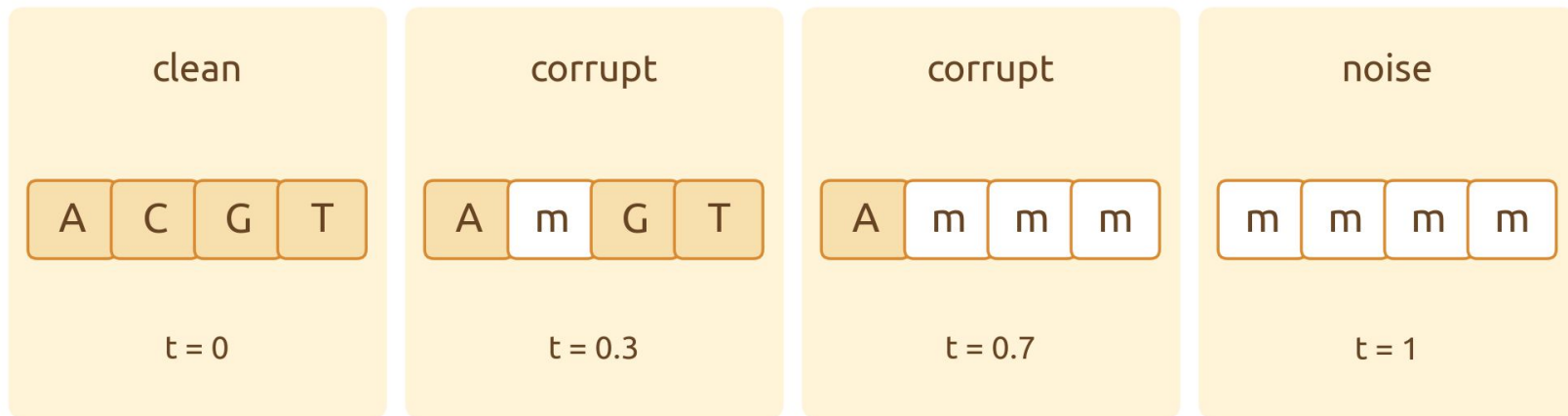
The probability of the entire corrupted sequence z given the clean sequence x , $q_t(z \mid x)$, equals the product over positions $i = 1$ through L of the probability of noisy token z^i given clean token x^i , $q_t(z^i \mid x^i)$.

x is the clean sequence; z is the noisy sequence; i indexes positions.

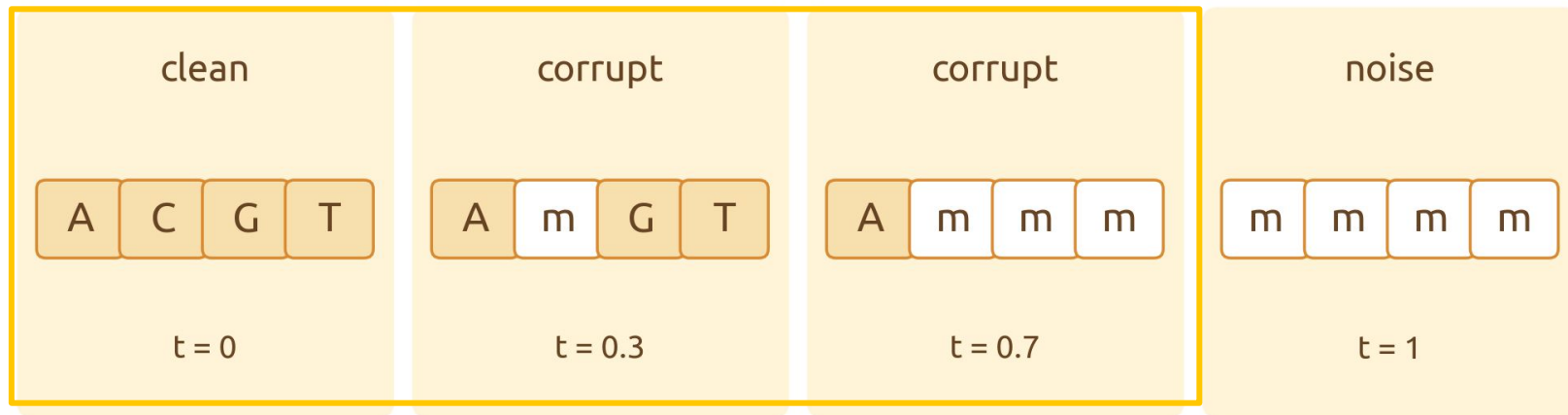
The corruptions are independent conditional on x . The data distribution can still contain strong dependencies between positions.

Once a token is masked, it stays masked during corruption

Once a token is masked, it stays masked during corruption

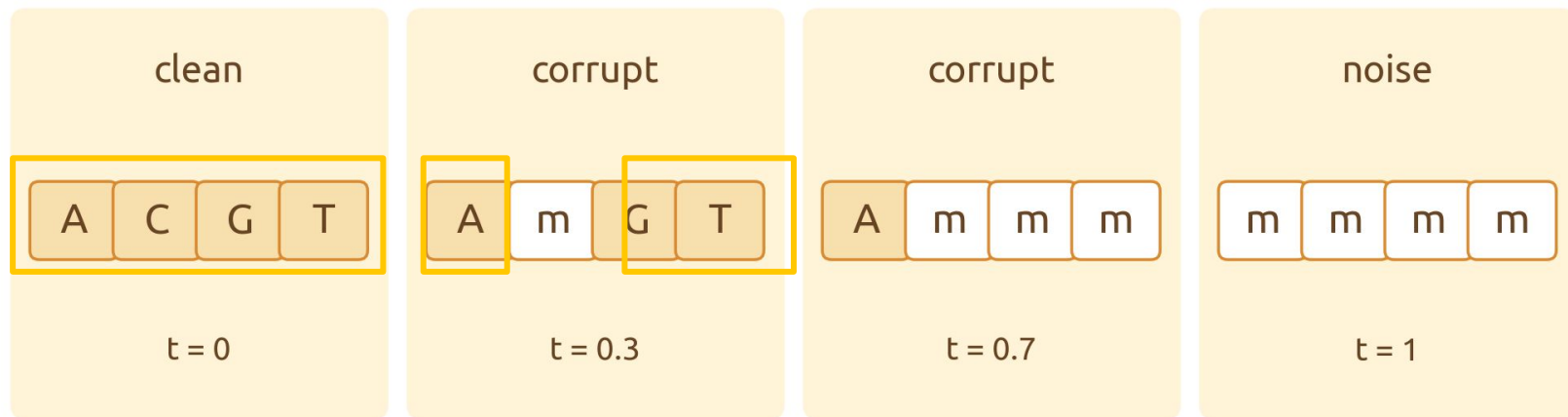


Once a token is masked, it stays masked during corruption



Each column begins with the same clean sequence. Increasing time erases additional positions.

Once a token is masked, it stays masked during corruption



Each column begins with the same clean sequence. Increasing time erases additional positions.

Every visible token still has its original identity, which makes the reverse posterior especially simple.

Now connect the same token at two noise levels

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

The probability $\Pr(Z_t = a \mid a)$ of retaining clean token a at time t equals its survival probability $\Pr(Z_s = a \mid a)$ at earlier time s times the probability $\Pr(Z_t = a \mid Z_s = a)$ of surviving the interval.

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

Substitute the two survival probabilities.

$$\alpha_t = \alpha_s \Pr(Z_t = a \mid Z_s = a)$$

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

Substitute the two survival probabilities.

$$\alpha_t = \alpha_s \Pr(Z_t = a \mid Z_s = a)$$

The later survival probability α_t equals earlier survival probability α_s multiplied by the conditional probability $\Pr(Z_t = a \mid Z_s = a)$ that clean token a remains visible from s to t .

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

Substitute the two survival probabilities.

$$\alpha_t = \alpha_s \Pr(Z_t = a \mid Z_s = a)$$

Take the complement.

$$\Pr(Z_t = m \mid Z_s = a) = 1 - \alpha_t / \alpha_s$$

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

Substitute the two survival probabilities.

$$\alpha_t = \alpha_s \Pr(Z_t = a \mid Z_s = a)$$

Take the complement.

$$\Pr(Z_t = m \mid Z_s = a) = 1 - \alpha_t / \alpha_s$$

The probability $\Pr(Z_t = m \mid Z_s = a)$ of masking visible token a between s and t equals one minus later survival probability α_t divided by earlier survival probability α_s .

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

Substitute the two survival probabilities.

$$\alpha_t = \alpha_s \Pr(Z_t = a \mid Z_s = a)$$

Take the complement.

$$\Pr(Z_t = m \mid Z_s = a) = 1 - \alpha_t / \alpha_s$$

The probability $\Pr(Z_t = m \mid Z_s = a)$ of masking visible token a between s and t equals one minus later survival probability α_t divided by earlier survival probability α_s .

Assume $s < t$ and $\alpha_s > 0$.

Now connect the same token at two noise levels

Survival to t requires survival to s first.

$$\Pr(Z_t = a \mid a) = \Pr(Z_s = a \mid a) \Pr(Z_t = a \mid Z_s = a)$$

Substitute the two survival probabilities.

$$\alpha_t = \alpha_s \Pr(Z_t = a \mid Z_s = a)$$

Take the complement.

$$\Pr(Z_t = m \mid Z_s = a) = 1 - \alpha_t / \alpha_s$$

The probability $\Pr(Z_t = m \mid Z_s = a)$ of masking visible token a between s and t equals one minus later survival probability α_t divided by earlier survival probability α_s .

Assume $s < t$ and $\alpha_s > 0$. The interval rule is determined by the marginal schedule.

To generate, we need to run that transition backward

To generate, we need to run that transition backward

Start with Bayes' rule.

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a, Z_t = m \mid a)}{q(Z_t = m \mid a)}$$

To generate, we need to run that transition backward

Start with Bayes' rule.

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a, Z_t = m \mid a)}{q(Z_t = m \mid a)}$$

The reveal probability $q(Z_s = a \mid Z_t = m, a)$ equals the probability $q(Z_s = a, Z_t = m \mid a)$ of being visible earlier and masked now, divided by the probability $q(Z_t = m \mid a)$ of being masked now, given clean token a .

To generate, we need to run that transition backward

Start with Bayes' rule.

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a, Z_t = m \mid a)}{q(Z_t = m \mid a)}$$

Factor the numerator along the forward path.

$$= \frac{\alpha_s(1 - \alpha_t/\alpha_s)}{1 - \alpha_t}$$

To generate, we need to run that transition backward

Start with Bayes' rule.

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a, Z_t = m \mid a)}{q(Z_t = m \mid a)}$$

Factor the numerator along the forward path.

$$= \frac{\alpha_s(1 - \alpha_t/\alpha_s)}{1 - \alpha_t}$$

The reveal probability $q(Z_s = a \mid Z_t = m, a)$ equals earlier survival α_s times the interval masking probability one minus α_t divided by α_s , with that product divided by current masking probability one minus α_t .

To generate, we need to run that transition backward

Start with Bayes' rule.

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a, Z_t = m \mid a)}{q(Z_t = m \mid a)}$$

Factor the numerator along the forward path.

$$= \frac{\alpha_s(1 - \alpha_t/\alpha_s)}{1 - \alpha_t}$$

Cancel the survival factor.

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

To generate, we need to run that transition backward

Start with Bayes' rule.

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a, Z_t = m \mid a)}{q(Z_t = m \mid a)}$$

Factor the numerator along the forward path.

$$= \frac{\alpha_s(1 - \alpha_t/\alpha_s)}{1 - \alpha_t}$$

Cancel the survival factor.

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The reveal probability $q(Z_s = a \mid Z_t = m, a)$ for clean token a equals earlier survival α_s minus later survival α_t , divided by current masking probability one minus α_t .

Generation reverses masking by revealing clean tokens

Start with Bayes' rule.

$$q(Z_s = a \mid Z_t = m, a) = \frac{q(Z_s = a, Z_t = m \mid a)}{q(Z_t = m \mid a)}$$

Factor the numerator along the forward path.

$$= \frac{\alpha_s(1 - \alpha_t/\alpha_s)}{1 - \alpha_t}$$

Cancel the survival factor.

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The reveal probability $q(Z_s = a \mid Z_t = m, a)$ for clean token a equals earlier survival α_s minus later survival α_t , divided by current masking probability one minus α_t .

We condition on tokens that are masked now, then ask which were still visible earlier.

A masked token can become visible in the reverse step

A masked token can become visible in the reverse step

Count paths that are visible at s .

$$q(Z_s = a \mid a) = \alpha_s$$

A masked token can become visible in the reverse step

Count paths that are visible at s .

$$q(Z_s = a \mid a) = \alpha_s$$

The probability $q(Z_s = a \mid a)$ that clean token a is still visible at earlier time s equals its survival probability α_s .

A masked token can become visible in the reverse step

Count paths that are visible at s.

$$q(Z_s = a \mid a) = \alpha_s$$

Keep only paths that mask before t.

$$q(Z_s = a, Z_t = m \mid a) = \alpha_s(1 - \alpha_t/\alpha_s) = \alpha_s - \alpha_t$$

A masked token can become visible in the reverse step

Count paths that are visible at s .

$$q(Z_s = a \mid a) = \alpha_s$$

Keep only paths that mask before t .

$$q(Z_s = a, Z_t = m \mid a) = \alpha_s(1 - \alpha_t/\alpha_s) = \alpha_s - \alpha_t$$

The probability $q(Z_s = a, Z_t = m \mid a)$ of surviving to s and being masked by t equals earlier survival α_s times one minus the survival ratio α_t/α_s , which simplifies to α_s minus α_t .

A masked token can become visible in the reverse step

Count paths that are visible at s.

$$q(Z_s = a \mid a) = \alpha_s$$

Keep only paths that mask before t.

$$q(Z_s = a, Z_t = m \mid a) = \alpha_s(1 - \alpha_t/\alpha_s) = \alpha_s - \alpha_t$$

Normalize within the currently masked paths.

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

A masked token can become visible in the reverse step

Count paths that are visible at s .

$$q(Z_s = a \mid a) = \alpha_s$$

Keep only paths that mask before t .

$$q(Z_s = a, Z_t = m \mid a) = \alpha_s(1 - \alpha_t/\alpha_s) = \alpha_s - \alpha_t$$

Normalize within the currently masked paths.

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The reveal probability $q(Z_s = a \mid Z_t = m, a)$ equals the visible-then-masked probability α_s minus α_t divided by current mask probability one minus α_t , given clean token a .

A masked token can become visible in the reverse step

Count paths that are visible at s .

$$q(Z_s = a \mid a) = \alpha_s$$

Keep only paths that mask before t .

$$q(Z_s = a, Z_t = m \mid a) = \alpha_s(1 - \alpha_t/\alpha_s) = \alpha_s - \alpha_t$$

Normalize within the currently masked paths.

$$q(Z_s = a \mid Z_t = m, a) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The reveal probability $q(Z_s = a \mid Z_t = m, a)$ equals the visible-then-masked probability α_s minus α_t divided by current mask probability one minus α_t , given clean token a .

We count paths that become masked during the interval, then divide by all paths that are masked now.

The remaining probability keeps that token masked

The remaining probability keeps that token masked

A currently masked token either reveals or stays.

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a)$$

The remaining probability keeps that token masked

A currently masked token either reveals or stays.

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a)$$

The probability $q(Z_s = m \mid Z_t = m, a)$ of remaining masked at the earlier time equals one minus the probability $q(Z_s = a \mid Z_t = m, a)$ of revealing clean token a .

The remaining probability keeps that token masked

A currently masked token either reveals or stays.

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a)$$

Substitute the reveal probability.

$$= 1 - \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The remaining probability keeps that token masked

A currently masked token either reveals or stays.

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a)$$

Substitute the reveal probability.

$$= 1 - \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The remaining-mask probability $q(Z_s = m \mid Z_t = m, a)$ equals one minus the reveal fraction, whose numerator is earlier survival α_s minus later survival α_t and whose denominator is current masking probability one minus α_t .

The remaining probability keeps that token masked

A currently masked token either reveals or stays.

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a)$$

Substitute the reveal probability.

$$= 1 - \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

Combine the numerator terms.

$$q(Z_s = m \mid Z_t = m, a) = \frac{1 - \alpha_s}{1 - \alpha_t}$$

The remaining probability keeps that token masked

A currently masked token either reveals or stays.

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a)$$

Substitute the reveal probability.

$$= 1 - \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

Combine the numerator terms.

$$q(Z_s = m \mid Z_t = m, a) = \frac{1 - \alpha_s}{1 - \alpha_t}$$

The remaining-mask probability $q(Z_s = m \mid Z_t = m, a)$ equals earlier masking probability one minus α_s divided by current masking probability one minus α_t , given clean token a and current mask m .

The remaining probability keeps that token masked

A currently masked token either reveals or stays.

$$q(Z_s = m \mid Z_t = m, a) = 1 - q(Z_s = a \mid Z_t = m, a)$$

Substitute the reveal probability.

$$= 1 - \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

Combine the numerator terms.

$$q(Z_s = m \mid Z_t = m, a) = \frac{1 - \alpha_s}{1 - \alpha_t}$$

The remaining-mask probability $q(Z_s = m \mid Z_t = m, a)$ equals earlier masking probability one minus α_s divided by current masking probability one minus α_t , given clean token a and current mask m .

Adding the reveal and remaining-mask probabilities gives exactly one.

We still need to predict which clean token to reveal

We still need to predict which clean token to reveal

$$d_{\theta}^i(a \mid z_t, t) \approx q(x_0^i = a \mid z_t)$$

We still need to predict which clean token to reveal

$$d_{\theta}^i(a \mid z_t, t) \approx q(x_0^i = a \mid z_t)$$

The denoiser probability $d_{\theta}^i(a \mid z_t, t)$ estimates posterior $q(x_0^i = a \mid z_t)$, the probability that clean token x_0^i at position i was a , given the full noisy sequence z_t , using parameters θ and noise time t .

We still need to predict which clean token to reveal

$$d_{\theta}^i(a \mid z_t, t) \approx q(x_0^i = a \mid z_t)$$

The denoiser probability $d_{\theta}^i(a \mid z_t, t)$ estimates posterior $q(x_0^i = a \mid z_t)$, the probability that clean token x_0^i at position i was a , given the full noisy sequence z_t , using parameters θ and noise time t .

θ denotes network parameters; d_{θ}^i is a categorical distribution over clean tokens.

We still need to predict which clean token to reveal

$$d_{\theta}^i(a \mid z_t, t) \approx q(x_0^i = a \mid z_t)$$

The denoiser probability $d_{\theta}^i(a \mid z_t, t)$ estimates posterior $q(x_0^i = a \mid z_t)$, the probability that clean token x_0^i at position i was a , given the full noisy sequence z_t , using parameters θ and noise time t .

θ denotes network parameters; d_{θ}^i is a categorical distribution over clean tokens.

We use context from every position to predict the missing token.

We average the reverse step over that prediction

We average the reverse step over that prediction

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

We average the reverse step over that prediction

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

The learned reveal probability $p_{\theta}(Z_s^i = a \mid z_t)$ sums over clean tokens b the reverse probability $q(Z_s^i = a \mid z_t^i = m, b)$, given mask m at position i , multiplied by the denoiser's clean-token probability $d_{\theta}^i(b \mid z_t)$ from noisy sequence z_t .

We average the reverse step over that prediction

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

A revealed base must equal its clean identity.

$$= \sum_b r_{s,t} \mathbf{1}\{a = b\} d_{\theta}^i(b \mid z_t)$$

We average the reverse step over that prediction

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

A revealed base must equal its clean identity.

$$= \sum_b r_{s,t} \mathbf{1}\{a = b\} d_{\theta}^i(b \mid z_t)$$

The learned reveal probability $p_{\theta}(Z_s^i = a \mid z_t)$ sums over clean candidates b the reveal probability $r_{s,t}$ times the indicator that a equals b , multiplied by predicted clean-token probability $d_{\theta}^i(b \mid z_t)$, so only b equal to a contributes.

We average the reverse step over that prediction

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

A revealed base must equal its clean identity.

$$= \sum_b r_{s,t} \mathbf{1}\{a = b\} d_{\theta}^i(b \mid z_t)$$

Evaluate the sum.

$$p_{\theta}(Z_s^i = a \mid z_t) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} d_{\theta}^i(a \mid z_t)$$

We average the reverse step over that prediction

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

A revealed base must equal its clean identity.

$$= \sum_b r_{s,t} \mathbf{1}\{a = b\} d_{\theta}^i(b \mid z_t)$$

Evaluate the sum.

$$p_{\theta}(Z_s^i = a \mid z_t) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} d_{\theta}^i(a \mid z_t)$$

The learned reveal probability $p_{\theta}(Z_s^i = a \mid z_t)$ equals earlier survival α_s minus later survival α_t , divided by masking probability one minus α_t , multiplied by the denoiser's probability $d_{\theta}^i(a \mid z_t)$ that token a belongs at position i .

Predict clean-token probabilities with the denoiser

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

A revealed base must equal its clean identity.

$$= \sum_b r_{s,t} \mathbf{1}\{a = b\} d_{\theta}^i(b \mid z_t)$$

Evaluate the sum.

$$p_{\theta}(Z_s^i = a \mid z_t) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} d_{\theta}^i(a \mid z_t)$$

The learned reveal probability $p_{\theta}(Z_s^i = a \mid z_t)$ equals earlier survival α_s minus later survival α_t , divided by masking probability one minus α_t , multiplied by the denoiser's probability $d_{\theta}^i(a \mid z_t)$ that token a belongs at position i .

First decide whether to reveal the position.

Predict clean-token probabilities with the denoiser

Average over the unknown clean identity.

$$p_{\theta}(Z_s^i = a \mid z_t) = \sum_b q(Z_s^i = a \mid z_t^i = m, b) d_{\theta}^i(b \mid z_t)$$

A revealed base must equal its clean identity.

$$= \sum_b r_{s,t} \mathbf{1}\{a = b\} d_{\theta}^i(b \mid z_t)$$

Evaluate the sum.

$$p_{\theta}(Z_s^i = a \mid z_t) = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} d_{\theta}^i(a \mid z_t)$$

The learned reveal probability $p_{\theta}(Z_s^i = a \mid z_t)$ equals earlier survival α_s minus later survival α_t , divided by masking probability one minus α_t , multiplied by the denoiser's probability $d_{\theta}^i(a \mid z_t)$ that token a belongs at position i .

First decide whether to reveal the position. If it is revealed, sample its DNA letter.

Let's calculate one complete reverse step

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Let's calculate one complete reverse step

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Among positions still masked at time t , the reveal probability r is the recovered survival mass $\alpha_s - \alpha_t$ divided by mask mass $1 - \alpha_t$, giving 0.5, so half are expected to become visible in this step.

Now divide the reveal probability between A and C

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Suppose the denoiser predicts A with 0.6 and C with 0.4.

$$p(A \mid z_t) = 0.5(0.6) = 0.3, \quad p(C \mid z_t) = 0.5(0.4) = 0.2$$

Now divide the reveal probability between A and C

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Suppose the denoiser predicts A with 0.6 and C with 0.4.

$$p(A \mid z_t) = 0.5(0.6) = 0.3, \quad p(C \mid z_t) = 0.5(0.4) = 0.2$$

We split that reveal mass using the denoiser's prediction, so $p(A \mid z_t) = 0.3$ and $p(C \mid z_t) = 0.2$; the more likely clean token receives more of the available reveal probability.

The remaining probability keeps the token masked

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Suppose the denoiser predicts A with 0.6 and C with 0.4.

$$p(A \mid z_t) = 0.5(0.6) = 0.3, \quad p(C \mid z_t) = 0.5(0.4) = 0.2$$

Include the probability of keeping the mask.

$$p(A, C, G, T, m \mid z_t) = (0.3, 0.2, 0, 0, 0.5), \quad \sum_b p(b \mid z_t) = 1$$

The remaining probability keeps the token masked

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Suppose the denoiser predicts A with 0.6 and C with 0.4.

$$p(A \mid z_t) = 0.5(0.6) = 0.3, \quad p(C \mid z_t) = 0.5(0.4) = 0.2$$

Include the probability of keeping the mask.

$$p(A, C, G, T, m \mid z_t) = (0.3, 0.2, 0, 0, 0.5), \quad \sum_b p(b \mid z_t) = 1$$

The reverse distribution $p(A, C, G, T, m \mid z_t)$ leaves probability 0.5 at the mask and allocates the other 0.5 to A and C, accounting for every possible next outcome.

We can now sample the next token from this distribution

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Suppose the denoiser predicts A with 0.6 and C with 0.4.

$$p(A \mid z_t) = 0.5(0.6) = 0.3, \quad p(C \mid z_t) = 0.5(0.4) = 0.2$$

Include the probability of keeping the mask.

$$p(A, C, G, T, m \mid z_t) = (0.3, 0.2, 0, 0, 0.5), \quad \sum_b p(b \mid z_t) = 1$$

The reverse distribution $p(A, C, G, T, m \mid z_t)$ leaves probability 0.5 at the mask and allocates the other 0.5 to A and C, accounting for every possible next outcome.

Half the probability reveals a token.

We can now sample the next token from this distribution

Use earlier survival 0.7 and later survival 0.4.

$$r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t} = \frac{0.7 - 0.4}{1 - 0.4} = \frac{0.3}{0.6} = 0.5$$

Suppose the denoiser predicts A with 0.6 and C with 0.4.

$$p(A \mid z_t) = 0.5(0.6) = 0.3, \quad p(C \mid z_t) = 0.5(0.4) = 0.2$$

Include the probability of keeping the mask.

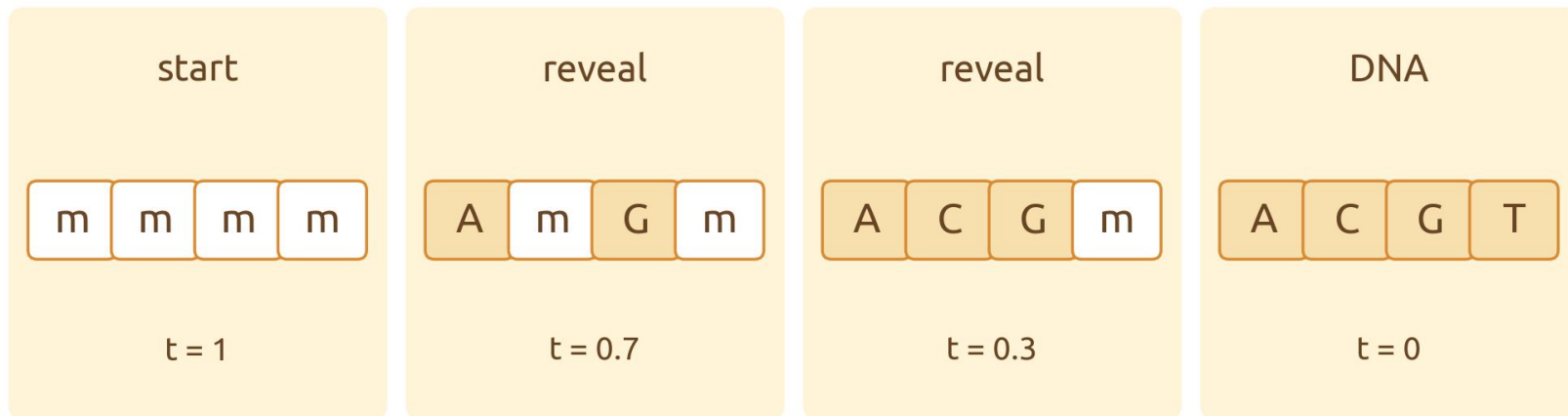
$$p(A, C, G, T, m \mid z_t) = (0.3, 0.2, 0, 0, 0.5), \quad \sum_b p(b \mid z_t) = 1$$

The reverse distribution $p(A, C, G, T, m \mid z_t)$ leaves probability 0.5 at the mask and allocates the other 0.5 to A and C, accounting for every possible next outcome.

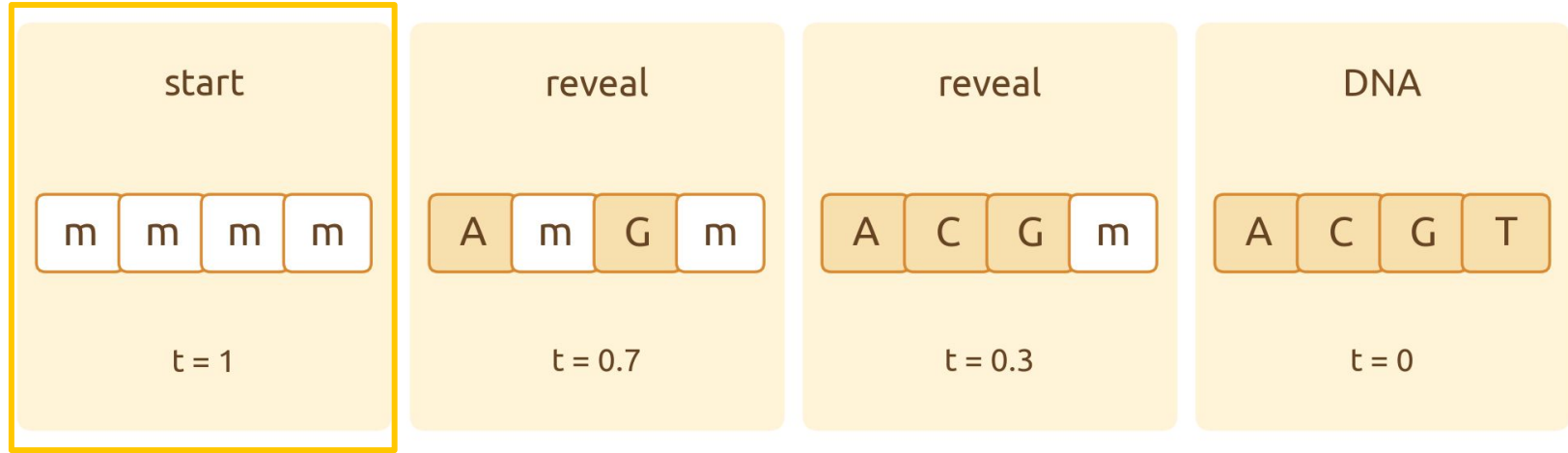
Half the probability reveals a token. The other half lets us reconsider this position using the next step's context.

Let's follow these reverse decisions through a full sequence

Let's follow these reverse decisions through a full sequence

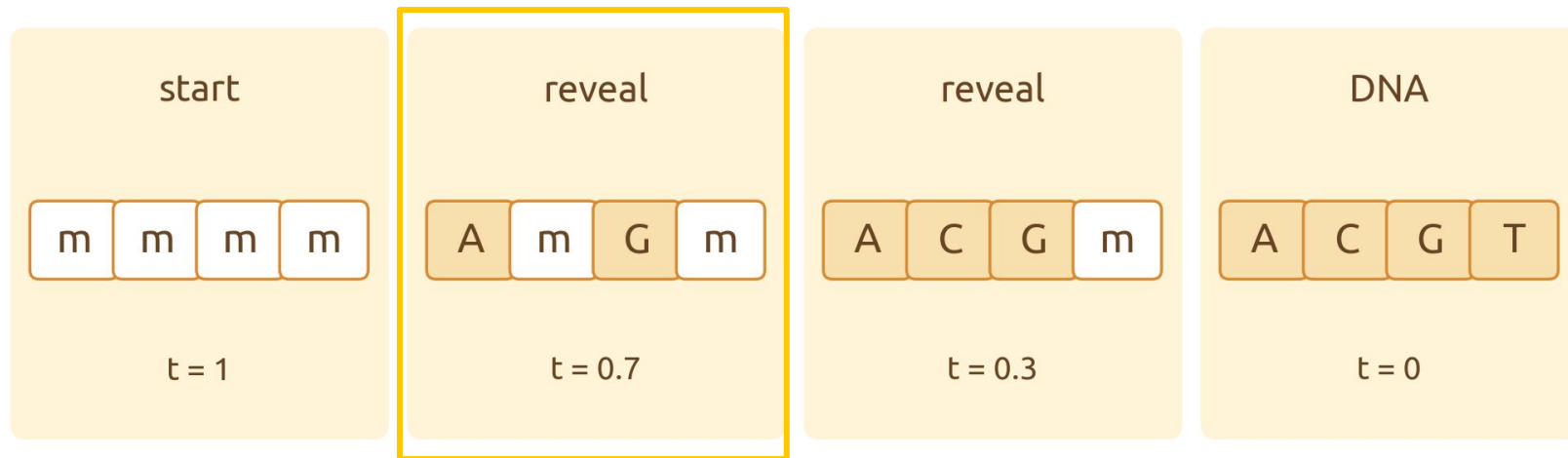


Start with every position masked



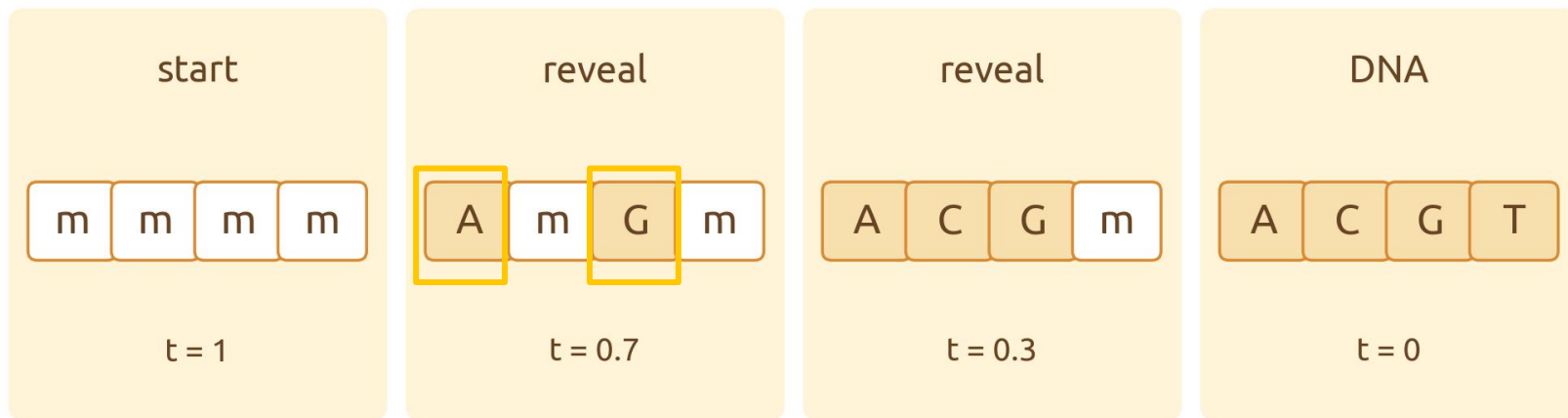
Start from mmmm at $t = 1$. Every DNA position is masked.

Let's follow these reverse decisions through a full sequence



First draw whether the masked position becomes visible.

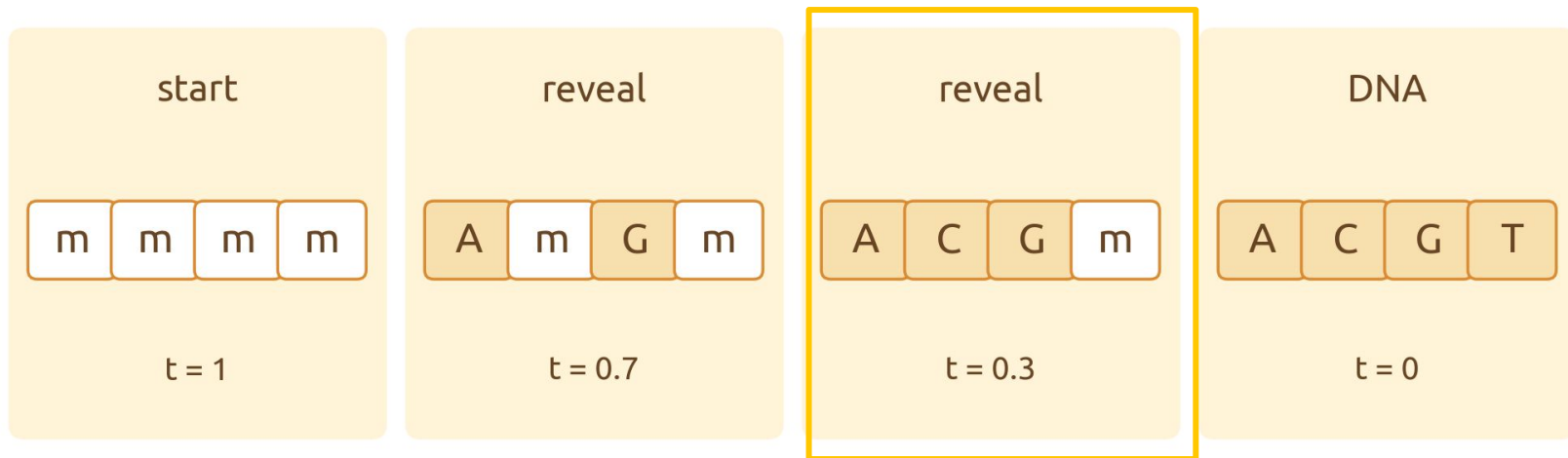
Let's follow these reverse decisions through a full sequence



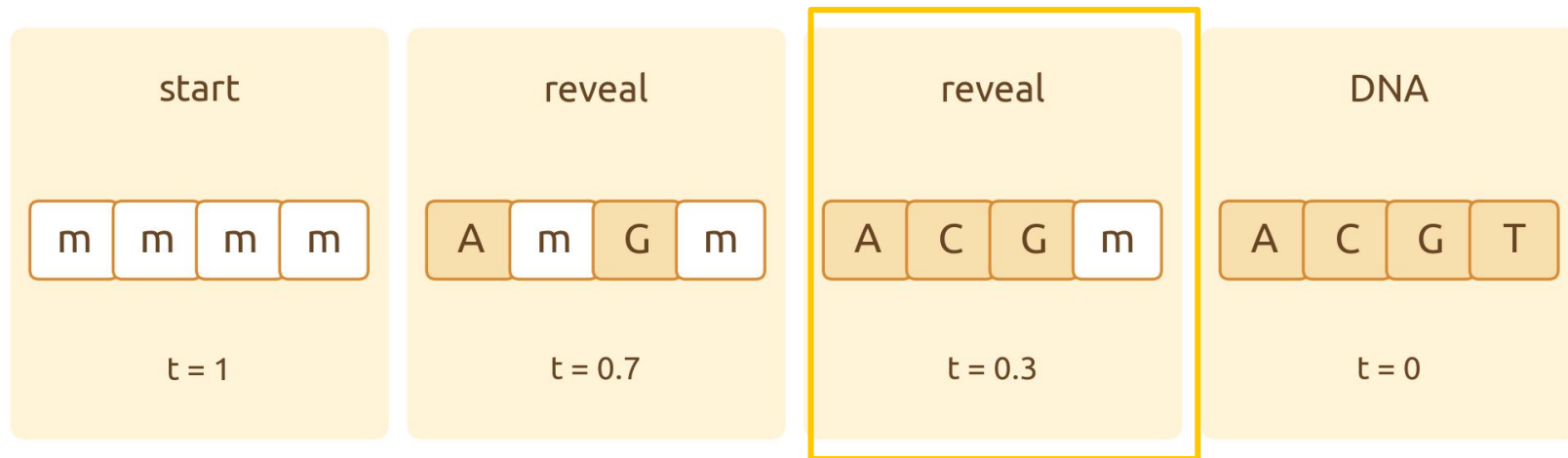
First draw whether the masked position becomes visible.

If it does, sample its clean identity from the denoiser. The outgoing probabilities sum to one.

Later steps reveal more letters using the visible context

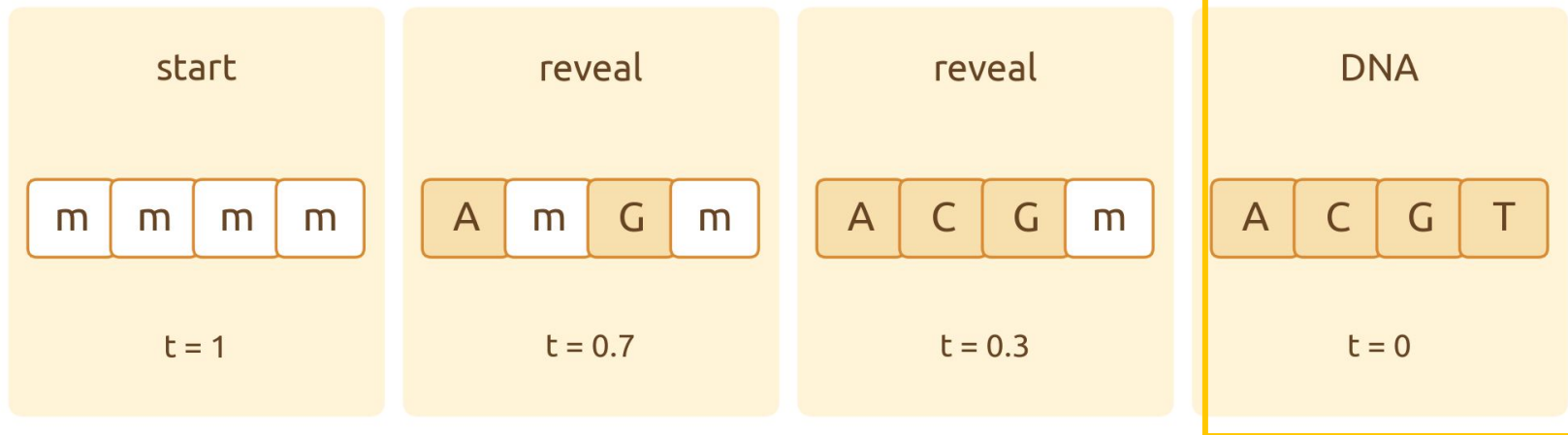


Later steps reveal more letters using the visible context

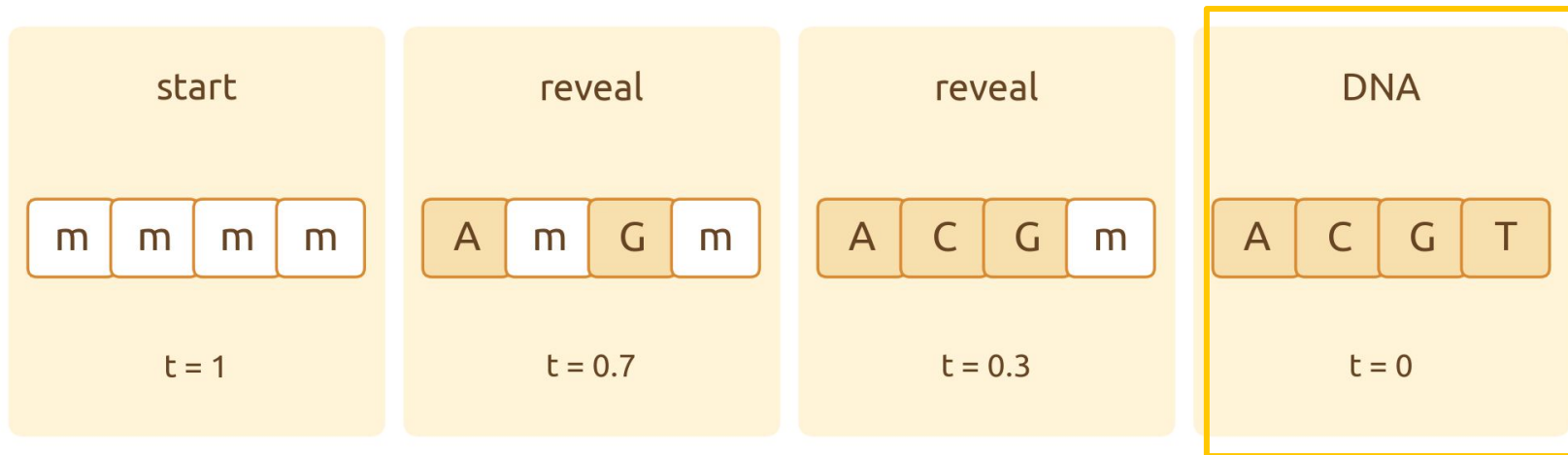


At $t = 0.3$, the next sampled reveal inserts C at position 2. The letters A and G remain fixed.

The last step reveals every remaining mask



The last step reveals every remaining mask



At $t = 0$, every remaining mask is revealed. Here the sampled final letter is T, giving ACGT.

To compare the model with the target, we need KL divergence

To compare the model with the target, we need KL divergence

We need to compare two probability distributions over the same outcomes.

$$D_{\text{KL}}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0$$

To compare the model with the target, we need KL divergence

We need to compare two probability distributions over the same outcomes.

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0$$

The divergence $D_{\text{KL}}(p \parallel q)$ equals the sum over outcomes x of reference probability $p(x)$ times the logarithm of $p(x)$ divided by comparison probability $q(x)$, and is nonnegative.

To compare the model with the target, we need KL divergence

We need to compare two probability distributions over the same outcomes.

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \geq 0$$

The divergence $D_{\text{KL}}(p \parallel q)$ equals the sum over outcomes x of reference probability $p(x)$ times the logarithm of $p(x)$ divided by comparison probability $q(x)$, and is nonnegative.

KL is zero exactly when the distributions agree on their support. If q assigns zero probability where p is positive, KL is infinite.

Let's calculate the mismatch between two predictions

Let's calculate the mismatch between two predictions

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

Let's calculate the mismatch between two predictions

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

The ratio q/p is above one at the token the model underestimates and below one at the token it overestimates, identifying the direction of each probability mismatch.

Each token contributes its weighted log ratio

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

Weight each target-to-model log ratio by its target probability.

$$0.75 \log 1.5 \approx 0.30410, \quad 0.25 \log 0.5 \approx -0.17329$$

Each token contributes its weighted log ratio

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

Weight each target-to-model log ratio by its target probability.

$$0.75 \log 1.5 \approx 0.30410, \quad 0.25 \log 0.5 \approx -0.17329$$

The target-weighted log contributions have opposite signs, so an individual term can be negative even though the complete KL divergence is nonnegative.

Adding the contributions gives the KL divergence

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

Weight each target-to-model log ratio by its target probability.

$$0.75 \log 1.5 \approx 0.30410, \quad 0.25 \log 0.5 \approx -0.17329$$

Adding the contributions gives the KL divergence

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

Weight each target-to-model log ratio by its target probability.

$$0.75 \log 1.5 \approx 0.30410, \quad 0.25 \log 0.5 \approx -0.17329$$

Add the contributions to obtain the full divergence.

$$D_{\text{KL}}(q||p) = 0.30410 - 0.17329 \approx 0.13081 \text{ nats}$$

Adding the contributions gives the KL divergence

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

Weight each target-to-model log ratio by its target probability.

$$0.75 \log 1.5 \approx 0.30410, \quad 0.25 \log 0.5 \approx -0.17329$$

Add the contributions to obtain the full divergence.

$$D_{\text{KL}}(q||p) = 0.30410 - 0.17329 \approx 0.13081 \text{ nats}$$

The full divergence $D_{\text{KL}}(q || p)$ is approximately 0.13081 nats after adding both contributions, quantifying the total mismatch between this target and model.

Matching the target would reduce this mismatch to zero

Choose target probabilities (0.75, 0.25) and model probabilities (0.5, 0.5).

$$q = (0.75, 0.25), \quad p = (0.5, 0.5), \quad q/p = (1.5, 0.5)$$

Weight each target-to-model log ratio by its target probability.

$$0.75 \log 1.5 \approx 0.30410, \quad 0.25 \log 0.5 \approx -0.17329$$

Add the contributions to obtain the full divergence.

$$D_{\text{KL}}(q||p) = 0.30410 - 0.17329 \approx 0.13081 \text{ nats}$$

The full divergence $D_{\text{KL}}(q || p)$ is approximately 0.13081 nats after adding both contributions, quantifying the total mismatch between this target and model. If the model prediction p matched q , both ratios would be one and the divergence would be zero.

To train this prediction, begin with the path probability

To train this prediction, begin with the path probability

The marginal sums over all noisy paths.

$$p_{\theta}(x_0) = \sum_{x_{1:N}} p_{\theta}(x_{0:N})$$

To train this prediction, begin with the path probability

The marginal sums over all noisy paths.

$$p_{\theta}(x_0) = \sum_{x_{1:N}} p_{\theta}(x_{0:N})$$

The probability $p_{\theta}(x_0)$ of clean sequence x_0 equals the sum over every noisy path $x_{1:N}$ of joint model probability $p_{\theta}(x_{0:N})$, where the full path runs from clean step zero to noise step N .

To train this prediction, begin with the path probability

The marginal sums over all noisy paths.

$$p_{\theta}(x_0) = \sum_{x_{1:N}} p_{\theta}(x_{0:N})$$

Multiply and divide by a tractable forward law.

$$= \sum_{x_{1:N}} q(x_{1:N} \mid x_0) \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}$$

To train this prediction, begin with the path probability

The marginal sums over all noisy paths.

$$p_{\theta}(x_0) = \sum_{x_{1:N}} p_{\theta}(x_{0:N})$$

Multiply and divide by a tractable forward law.

$$= \sum_{x_{1:N}} q(x_{1:N} \mid x_0) \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}$$

The clean-sequence probability $p_{\theta}(x_0)$ equals the sum over noisy paths $x_{1:N}$ of forward path probability $q(x_{1:N} \mid x_0)$, multiplied by joint model probability $p_{\theta}(x_{0:N})$ divided by that same forward probability.

To train this prediction, begin with the path probability

The marginal sums over all noisy paths.

$$p_{\theta}(x_0) = \sum_{x_{1:N}} p_{\theta}(x_{0:N})$$

Multiply and divide by a tractable forward law.

$$= \sum_{x_{1:N}} q(x_{1:N} \mid x_0) \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}$$

Recognize an expectation.

$$p_{\theta}(x_0) = \mathbb{E}_q \left[\frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)} \right]$$

To train this prediction, begin with the path probability

The marginal sums over all noisy paths.

$$p_{\theta}(x_0) = \sum_{x_{1:N}} p_{\theta}(x_{0:N})$$

Multiply and divide by a tractable forward law.

$$= \sum_{x_{1:N}} q(x_{1:N} \mid x_0) \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}$$

Recognize an expectation.

$$p_{\theta}(x_0) = \mathbb{E}_q \left[\frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)} \right]$$

The clean-sequence probability $p_{\theta}(x_0)$ equals the average under forward path law q of joint model probability $p_{\theta}(x_{0:N})$ divided by forward conditional probability $q(x_{1:N} \mid x_0)$.

To train this prediction, begin with the path probability

The marginal sums over all noisy paths.

$$p_{\theta}(x_0) = \sum_{x_{1:N}} p_{\theta}(x_{0:N})$$

Multiply and divide by a tractable forward law.

$$= \sum_{x_{1:N}} q(x_{1:N} \mid x_0) \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}$$

Recognize an expectation.

$$p_{\theta}(x_0) = \mathbb{E}_q \left[\frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)} \right]$$

The clean-sequence probability $p_{\theta}(x_0)$ equals the average under forward path law q of joint model probability $p_{\theta}(x_{0:N})$ divided by forward conditional probability $q(x_{1:N} \mid x_0)$.

Require the forward law to cover the relevant model paths. We can sample this auxiliary law.

Jensen's inequality will give us a trainable lower bound

Jensen's inequality will give us a trainable lower bound

A function is concave when its graph lies above each chord joining two points.

$$f(\lambda a + (1 - \lambda)b) \geq \lambda f(a) + (1 - \lambda)f(b) \quad \implies \quad f(\mathbb{E}W) \geq \mathbb{E}f(W)$$

Jensen's inequality will give us a trainable lower bound

A function is concave when its graph lies above each chord joining two points.

$$f(\lambda a + (1 - \lambda)b) \geq \lambda f(a) + (1 - \lambda)f(b) \implies f(\mathbb{E}W) \geq \mathbb{E}f(W)$$

For concave function f , its value $f(\lambda a + (1 - \lambda)b)$ at the weighted mean of a and b is at least $\lambda f(a)$ plus one minus λ times $f(b)$, giving $f(\mathbb{E}W) \geq \mathbb{E}[f(W)]$ when averaging random input W .

Jensen's inequality will give us a trainable lower bound

A function is concave when its graph lies above each chord joining two points.

$$f(\lambda a + (1 - \lambda)b) \geq \lambda f(a) + (1 - \lambda)f(b) \implies f(\mathbb{E}W) \geq \mathbb{E}f(W)$$

For concave function f , its value $f(\lambda a + (1 - \lambda)b)$ at the weighted mean of a and b is at least $\lambda f(a)$ plus one minus λ times $f(b)$, giving $f(\mathbb{E}W) \geq \mathbb{E}[f(W)]$ when averaging random input W .

Logarithm is concave. For W equal to 1 or 4 with equal probability, $\log(2.5) \approx 0.916$ exceeds $[\log(1) + \log(4)]/2 \approx 0.693$.

Let's check the bound with two possible path weights

Choose two equally likely positive path weights, 0.5 and 1.5.

$$P(W = 0.5) = P(W = 1.5) = 1/2$$

Let's check the bound with two possible path weights

Choose two equally likely positive path weights, 0.5 and 1.5.

$$P(W = 0.5) = P(W = 1.5) = 1/2$$

The positive weight W takes values 0.5 and 1.5 with equal probability, giving a simple example of uncertainty across possible paths.

First average the weights, then take the logarithm

Choose two equally likely positive path weights, 0.5 and 1.5.

$$P(W = 0.5) = P(W = 1.5) = 1/2$$

First average the weights and then take the logarithm.

$$\log \mathbb{E}[W] = \log \left[\frac{1}{2}(0.5) + \frac{1}{2}(1.5) \right] = \log 1 = 0$$

First average the weights, then take the logarithm

Choose two equally likely positive path weights, 0.5 and 1.5.

$$P(W = 0.5) = P(W = 1.5) = 1/2$$

First average the weights and then take the logarithm.

$$\log \mathbb{E}[W] = \log \left[\frac{1}{2}(0.5) + \frac{1}{2}(1.5) \right] = \log 1 = 0$$

The log of the mean weight, $\log \mathbb{E}[W]$, is zero because the low and high weights average to one before applying the logarithm.

Taking the logarithm first gives a smaller average

Choose two equally likely positive path weights, 0.5 and 1.5.

$$P(W = 0.5) = P(W = 1.5) = 1/2$$

First average the weights and then take the logarithm.

$$\log \mathbb{E}[W] = \log \left[\frac{1}{2}(0.5) + \frac{1}{2}(1.5) \right] = \log 1 = 0$$

Now take the logarithm first and average afterward.

$$\mathbb{E}[\log W] = \frac{1}{2} \log 0.5 + \frac{1}{2} \log 1.5 \approx -0.14384 \leq 0$$

Taking the logarithm first gives a smaller average

Choose two equally likely positive path weights, 0.5 and 1.5.

$$P(W = 0.5) = P(W = 1.5) = 1/2$$

First average the weights and then take the logarithm.

$$\log \mathbb{E}[W] = \log \left[\frac{1}{2}(0.5) + \frac{1}{2}(1.5) \right] = \log 1 = 0$$

Now take the logarithm first and average afterward.

$$\mathbb{E}[\log W] = \frac{1}{2} \log 0.5 + \frac{1}{2} \log 1.5 \approx -0.14384 \leq 0$$

The mean log weight $\mathbb{E}[\log W]$ is approximately -0.14384, below $\log \mathbb{E}[W]$, because averaging after the concave logarithm gives the lower bound.

Variation in the path weights leaves a gap in the bound

Choose two equally likely positive path weights, 0.5 and 1.5.

$$P(W = 0.5) = P(W = 1.5) = 1/2$$

First average the weights and then take the logarithm.

$$\log \mathbb{E}[W] = \log \left[\frac{1}{2}(0.5) + \frac{1}{2}(1.5) \right] = \log 1 = 0$$

Now take the logarithm first and average afterward.

$$\mathbb{E}[\log W] = \frac{1}{2} \log 0.5 + \frac{1}{2} \log 1.5 \approx -0.14384 \leq 0$$

The mean log weight $\mathbb{E}[\log W]$ is approximately -0.14384, below $\log \mathbb{E}[W]$, because averaging after the concave logarithm gives the lower bound.

The gap is about 0.14384. Equal weights would eliminate this gap, connecting bound tightness to path-weight variation.

Taking the logarithm gives us a lower bound to optimize

Taking the logarithm gives us a lower bound to optimize

Use the positive path weight.

$$W = \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}, \quad \log p_{\theta}(x_0) = \log \mathbb{E}_q W$$

Taking the logarithm gives us a lower bound to optimize

Use the positive path weight.

$$W = \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}, \quad \log p_{\theta}(x_0) = \log \mathbb{E}_q W$$

The path weight W equals joint model probability $p_{\theta}(x_{0:N})$ divided by forward conditional path probability $q(x_{1:N} \mid x_0)$, so clean-sequence log probability $\log p_{\theta}(x_0)$ equals the logarithm of average path weight $\mathbb{E}_q W$.

Taking the logarithm gives us a lower bound to optimize

Use the positive path weight.

$$W = \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}, \quad \log p_{\theta}(x_0) = \log \mathbb{E}_q W$$

Apply concavity of logarithm.

$$\log \mathbb{E}_q W \geq \mathbb{E}_q \log W$$

Taking the logarithm gives us a lower bound to optimize

Use the positive path weight.

$$W = \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}, \quad \log p_{\theta}(x_0) = \log \mathbb{E}_q W$$

Apply concavity of logarithm.

$$\log \mathbb{E}_q W \geq \mathbb{E}_q \log W$$

The logarithm $\log \mathbb{E}_q W$ of the average positive path weight is at least the average log weight $\mathbb{E}_q \log W$, with both averages taken over noisy paths sampled from forward law q .

Taking the logarithm gives us a lower bound to optimize

Use the positive path weight.

$$W = \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}, \quad \log p_{\theta}(x_0) = \log \mathbb{E}_q W$$

Apply concavity of logarithm.

$$\log \mathbb{E}_q W \geq \mathbb{E}_q \log W$$

Expand the log ratio.

$$\text{ELBO} = \mathbb{E}_q [\log p_{\theta}(x_{0:N}) - \log q(x_{1:N} \mid x_0)]$$

Taking the logarithm gives us a lower bound to optimize

Use the positive path weight.

$$W = \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}, \quad \log p_{\theta}(x_0) = \log \mathbb{E}_q W$$

Apply concavity of logarithm.

$$\log \mathbb{E}_q W \geq \mathbb{E}_q \log W$$

Expand the log ratio.

$$\text{ELBO} = \mathbb{E}_q [\log p_{\theta}(x_{0:N}) - \log q(x_{1:N} \mid x_0)]$$

The evidence lower bound ELBO equals the average under forward law q of joint path log probability $\log p_{\theta}(x_{0:N})$ minus forward conditional log probability $\log q(x_{1:N} \mid x_0)$, for clean sequence x_0 and noisy path $x_{1:N}$.

We can optimize this bound using sampled noisy paths

Use the positive path weight.

$$W = \frac{p_{\theta}(x_{0:N})}{q(x_{1:N} \mid x_0)}, \quad \log p_{\theta}(x_0) = \log \mathbb{E}_q W$$

Apply concavity of logarithm.

$$\log \mathbb{E}_q W \geq \mathbb{E}_q \log W$$

Expand the log ratio.

$$\text{ELBO} = \mathbb{E}_q [\log p_{\theta}(x_{0:N}) - \log q(x_{1:N} \mid x_0)]$$

The evidence lower bound ELBO equals the average under forward law q of joint path log probability $\log p_{\theta}(x_{0:N})$ minus forward conditional log probability $\log q(x_{1:N} \mid x_0)$, for clean sequence x_0 and noisy path $x_{1:N}$.

The expectation is tractable even when the model marginal requires an enormous path sum.

We can also identify the gap in this bound

We can also identify the gap in this bound

Expand the conditional-path KL.

$$D_{\text{KL}}(q||p_{\theta}(\cdot \mid x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} \mid x_0)}{p_{\theta}(x_{1:N} \mid x_0)}$$

We can also identify the gap in this bound

Expand the conditional-path KL.

$$D_{\text{KL}}(q \| p_{\theta}(\cdot \mid x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} \mid x_0)}{p_{\theta}(x_{1:N} \mid x_0)}$$

The path divergence $D_{\text{KL}}(q \parallel p_{\theta}(\cdot \mid x_0))$ equals the average under forward law q of the logarithm of forward path probability $q(x_{1:N} \mid x_0)$ divided by model posterior path probability $p_{\theta}(x_{1:N} \mid x_0)$, given clean sequence x_0 .

We can also identify the gap in this bound

Expand the conditional-path KL.

$$D_{\text{KL}}(q||p_{\theta}(\cdot | x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} | x_0)}{p_{\theta}(x_{1:N} | x_0)}$$

Replace the posterior with joint divided by marginal.

$$= \mathbb{E}_q[\log q - \log p_{\theta}(x_{0:N}) + \log p_{\theta}(x_0)]$$

We can also identify the gap in this bound

Expand the conditional-path KL.

$$D_{\text{KL}}(q \| p_{\theta}(\cdot \mid x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} \mid x_0)}{p_{\theta}(x_{1:N} \mid x_0)}$$

Replace the posterior with joint divided by marginal.

$$= \mathbb{E}_q [\log q - \log p_{\theta}(x_{0:N}) + \log p_{\theta}(x_0)]$$

The path divergence $D_{\text{KL}}(q \parallel p_{\theta}(\cdot \mid x_0))$ equals the forward-path average of $\log q(x_{1:N} \mid x_0)$, the forward conditional log probability, minus joint model log probability $\log p_{\theta}(x_{0:N})$, plus clean-sequence log probability $\log p_{\theta}(x_0)$.

We can also identify the gap in this bound

Expand the conditional-path KL.

$$D_{\text{KL}}(q||p_{\theta}(\cdot | x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} | x_0)}{p_{\theta}(x_{1:N} | x_0)}$$

Replace the posterior with joint divided by marginal.

$$= \mathbb{E}_q [\log q - \log p_{\theta}(x_{0:N}) + \log p_{\theta}(x_0)]$$

Identify the lower bound.

$$D_{\text{KL}}(q||p_{\theta}(\cdot | x_0)) = \log p_{\theta}(x_0) - \text{ELBO} \geq 0$$

We can also identify the gap in this bound

Expand the conditional-path KL.

$$D_{\text{KL}}(q||p_{\theta}(\cdot | x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} | x_0)}{p_{\theta}(x_{1:N} | x_0)}$$

Replace the posterior with joint divided by marginal.

$$= \mathbb{E}_q [\log q - \log p_{\theta}(x_{0:N}) + \log p_{\theta}(x_0)]$$

Identify the lower bound.

$$D_{\text{KL}}(q||p_{\theta}(\cdot | x_0)) = \log p_{\theta}(x_0) - \text{ELBO} \geq 0$$

The path divergence $D_{\text{KL}}(q || p_{\theta}(\cdot | x_0))$ equals clean-sequence log probability $\log p_{\theta}(x_0)$ minus evidence lower bound ELBO, and is at least zero because KL divergence is nonnegative.

We can also identify the gap in this bound

Expand the conditional-path KL.

$$D_{\text{KL}}(q \| p_{\theta}(\cdot \mid x_0)) = \mathbb{E}_q \log \frac{q(x_{1:N} \mid x_0)}{p_{\theta}(x_{1:N} \mid x_0)}$$

Replace the posterior with joint divided by marginal.

$$= \mathbb{E}_q [\log q - \log p_{\theta}(x_{0:N}) + \log p_{\theta}(x_0)]$$

Identify the lower bound.

$$D_{\text{KL}}(q \| p_{\theta}(\cdot \mid x_0)) = \log p_{\theta}(x_0) - \text{ELBO} \geq 0$$

The path divergence $D_{\text{KL}}(q \| p_{\theta}(\cdot \mid x_0))$ equals clean-sequence log probability $\log p_{\theta}(x_0)$ minus evidence lower bound ELBO, and is at least zero because KL divergence is nonnegative.

Equality holds exactly when the two conditional path laws agree on their support.

Now apply the constraints we imposed on the denoiser

Now apply the constraints we imposed on the denoiser

$$d_{\theta}^i(m \mid z_t) = 0, \quad z_t^i \neq m \implies d_{\theta}^i(a \mid z_t) = \mathbf{1}\{a = z_t^i\}$$

Now apply the constraints we imposed on the denoiser

$$d_{\theta}^i(m \mid z_t) = 0, \quad z_t^i \neq m \implies d_{\theta}^i(a \mid z_t) = \mathbf{1}\{a = z_t^i\}$$

The clean-token prediction $d_{\theta}^i(m \mid z_t)$ assigns zero probability to mask m , and at a visible position i the probability $d_{\theta}^i(a \mid z_t)$ equals the indicator that candidate a matches observed token z_t^i , preserving that token.

Now apply the constraints we imposed on the denoiser

$$d_{\theta}^i(m \mid z_t) = 0, \quad z_t^i \neq m \implies d_{\theta}^i(a \mid z_t) = \mathbf{1}\{a = z_t^i\}$$

The clean-token prediction $d_{\theta}^i(m \mid z_t)$ assigns zero probability to mask m , and at a visible position i the probability $d_{\theta}^i(a \mid z_t)$ equals the indicator that candidate a matches observed token z_t^i , preserving that token.

SUBS is the substitution parameterization. The neural network is needed at masked positions.

Now apply the constraints we imposed on the denoiser

$$d_{\theta}^i(m \mid z_t) = 0, \quad z_t^i \neq m \implies d_{\theta}^i(a \mid z_t) = \mathbf{1}\{a = z_t^i\}$$

The clean-token prediction $d_{\theta}^i(m \mid z_t)$ assigns zero probability to mask m , and at a visible position i the probability $d_{\theta}^i(a \mid z_t)$ equals the indicator that candidate a matches observed token z_t^i , preserving that token.

SUBS is the substitution parameterization. The neural network is needed at masked positions.

These constraints encode facts already implied by the corruption process, eliminating unnecessary prediction tasks.

The model and target assign the same mass to the mask

The model and target assign the same mass to the mask

$$q_s = (1 - r)\delta_m + r\delta_a, \quad p_{\theta,s} = (1 - r)\delta_m + rd_\theta, \quad r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The model and target assign the same mass to the mask

$$q_s = (1 - r)\delta_m + r\delta_a, \quad p_{\theta,s} = (1 - r)\delta_m + rd_\theta, \quad r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The exact reverse law q_s assigns probability one minus r to mask point mass δ_m and r to clean-token point mass δ_a , while learned law $p_{\theta,s}$ distributes the same reveal probability r through d_θ , with r equal to $(\alpha_s - \alpha_t)/(1 - \alpha_t)$.

The model and target assign the same mass to the mask

$$q_s = (1 - r)\delta_m + r\delta_a, \quad p_{\theta,s} = (1 - r)\delta_m + rd_\theta, \quad r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The exact reverse law q_s assigns probability one minus r to mask point mass δ_m and r to clean-token point mass δ_a , while learned law $p_{\theta,s}$ distributes the same reveal probability r through d_θ , with r equal to $(\alpha_s - \alpha_t)/(1 - \alpha_t)$.

δ_m and δ_a are point masses.

The model and target assign the same mass to the mask

$$q_s = (1 - r)\delta_m + r\delta_a, \quad p_{\theta,s} = (1 - r)\delta_m + rd_\theta, \quad r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The exact reverse law q_s assigns probability one minus r to mask point mass δ_m and r to clean-token point mass δ_a , while learned law $p_{\theta,s}$ distributes the same reveal probability r through d_θ , with r equal to $(\alpha_s - \alpha_t)/(1 - \alpha_t)$.

δ_m and δ_a are point masses. Both reverse distributions use the same schedule-derived reveal probability r .

The model and target assign the same mass to the mask

$$q_s = (1 - r)\delta_m + r\delta_a, \quad p_{\theta,s} = (1 - r)\delta_m + rd_\theta, \quad r = \frac{\alpha_s - \alpha_t}{1 - \alpha_t}$$

The exact reverse law q_s assigns probability one minus r to mask point mass δ_m and r to clean-token point mass δ_a , while learned law $p_{\theta,s}$ distributes the same reveal probability r through d_θ , with r equal to $(\alpha_s - \alpha_t)/(1 - \alpha_t)$.

δ_m and δ_a are point masses. Both reverse distributions use the same schedule-derived reveal probability r .

The only disagreement is how the learned model distributes probability among clean letters.

That shared mask term cancels from the KL divergence

That shared mask term cancels from the KL divergence

Write only the two nonzero target outcomes.

$$q = (1 - r)\delta_m + r\delta_a, \quad p = (1 - r)\delta_m + rd_\theta$$

That shared mask term cancels from the KL divergence

Write only the two nonzero target outcomes.

$$q = (1 - r)\delta_m + r\delta_a, \quad p = (1 - r)\delta_m + rd_\theta$$

The exact reverse distribution q assigns remaining-mask probability one minus r to point mass δ_m and reveal probability r to clean-token point mass δ_a , while learned distribution p assigns that same reveal probability to denoiser distribution d_θ .

That shared mask term cancels from the KL divergence

Write only the two nonzero target outcomes.

$$q = (1 - r)\delta_m + r\delta_a, \quad p = (1 - r)\delta_m + rd_\theta$$

Expand the categorical KL.

$$D_{\text{KL}}(q\|p) = (1 - r) \log \frac{1 - r}{1 - r} + r \log \frac{r}{rd_\theta(a)}$$

That shared mask term cancels from the KL divergence

Write only the two nonzero target outcomes.

$$q = (1 - r)\delta_m + r\delta_a, \quad p = (1 - r)\delta_m + rd_\theta$$

Expand the categorical KL.

$$D_{\text{KL}}(q||p) = (1 - r) \log \frac{1 - r}{1 - r} + r \log \frac{r}{rd_\theta(a)}$$

The reverse divergence $D_{\text{KL}}(q || p)$ equals mask probability one minus r times the log of identical mask probabilities divided by each other, plus reveal probability r times the log of r divided by r times predicted clean probability $d_\theta(a)$.

That shared mask term cancels from the KL divergence

Write only the two nonzero target outcomes.

$$q = (1 - r)\delta_m + r\delta_a, \quad p = (1 - r)\delta_m + rd_\theta$$

Expand the categorical KL.

$$D_{\text{KL}}(q\|p) = (1 - r) \log \frac{1 - r}{1 - r} + r \log \frac{r}{rd_\theta(a)}$$

Cancel the mask term and the common reveal factor.

$$D_{\text{KL}}(q\|p) = -r \log d_\theta(a)$$

That shared mask term cancels from the KL divergence

Write only the two nonzero target outcomes.

$$q = (1 - r)\delta_m + r\delta_a, \quad p = (1 - r)\delta_m + rd_\theta$$

Expand the categorical KL.

$$D_{\text{KL}}(q||p) = (1 - r) \log \frac{1 - r}{1 - r} + r \log \frac{r}{rd_\theta(a)}$$

Cancel the mask term and the common reveal factor.

$$D_{\text{KL}}(q||p) = -r \log d_\theta(a)$$

The reverse divergence $D_{\text{KL}}(q || p)$ equals reveal probability r multiplied by the negative logarithm of $d_\theta(a)$, the denoiser's predicted probability of the correct clean token a .

Only the revealed clean token contributes to this KL

Write only the two nonzero target outcomes.

$$q = (1 - r)\delta_m + r\delta_a, \quad p = (1 - r)\delta_m + rd_\theta$$

Expand the categorical KL.

$$D_{\text{KL}}(q\|p) = (1 - r) \log \frac{1 - r}{1 - r} + r \log \frac{r}{rd_\theta(a)}$$

Cancel the mask term and the common reveal factor.

$$D_{\text{KL}}(q\|p) = -r \log d_\theta(a)$$

The reverse divergence $D_{\text{KL}}(q \parallel p)$ equals reveal probability r multiplied by the negative logarithm of $d_\theta(a)$, the denoiser's predicted probability of the correct clean token a .
The full categorical reverse loss reduces to one weighted cross-entropy.

At a visible position, the entire KL term is zero

At a visible position, the entire KL term is zero

$$z_t^i = a \neq m \implies q(z_s^i \mid z_t^i, a) = p_\theta(z_s^i \mid z_t) = \delta_a \implies D_{\text{KL}} = 0$$

At a visible position, the entire KL term is zero

$$z_t^i = a \neq m \implies q(z_s^i \mid z_t^i, a) = p_\theta(z_s^i \mid z_t) = \delta_a \implies D_{\text{KL}} = 0$$

For observed token z_t^i equal to clean token a and unequal to mask m , both exact reverse law q and learned reverse law p_θ put probability one on a through point mass δ_a , so their KL divergence is zero.

At a visible position, the entire KL term is zero

$$z_t^i = a \neq m \implies q(z_s^i \mid z_t^i, a) = p_\theta(z_s^i \mid z_t) = \delta_a \implies D_{\text{KL}} = 0$$

For observed token z_t^i equal to clean token a and unequal to mask m , both exact reverse law q and learned reverse law p_θ put probability one on a through point mass δ_a , so their KL divergence is zero.

The equality uses carry-over preservation in SUBS.

At a visible position, the entire KL term is zero

$$z_t^i = a \neq m \implies q(z_s^i \mid z_t^i, a) = p_\theta(z_s^i \mid z_t) = \delta_a \implies D_{\text{KL}} = 0$$

For observed token z_t^i equal to clean token a and unequal to mask m , both exact reverse law q and learned reverse law p_θ put probability one on a through point mass δ_a , so their KL divergence is zero.

The equality uses carry-over preservation in SUBS.

Only masked positions need a loss contribution.

So we only need to train at masked positions

So we only need to train at masked positions

We can sample a clean sequence and corrupt it to any time without simulating the full trajectory.

So we only need to train at masked positions

We can sample a clean sequence and corrupt it to any time without simulating the full trajectory.

The training target is the original token at each masked position.

The remaining loss uses the correct clean-token probability

The remaining loss uses the correct clean-token probability

Define the error only at currently masked positions.

$$\ell_{\theta}(z_t, x_0) = \sum_{i: z_t^i = m} -\log d_{\theta}^i(x_0^i \mid z_t, t)$$

The remaining loss uses the correct clean-token probability

Define the error only at currently masked positions.

$$\ell_{\theta}(z_t, x_0) = \sum_{i: z_t^i = m} -\log d_{\theta}^i(x_0^i \mid z_t, t)$$

The masked-token loss $\ell_{\theta}(z_t, x_0)$ sums over positions i where noisy token z_t^i equals mask m the negative logarithm of $d_{\theta}^i(x_0^i \mid z_t, t)$, the predicted probability of true clean token x_0^i given noisy sequence z_t and time t .

The remaining loss uses the correct clean-token probability

Define the error only at currently masked positions.

$$\ell_{\theta}(z_t, x_0) = \sum_{i: z_t^i = m} -\log d_{\theta}^i(x_0^i \mid z_t, t)$$

Weight by the infinitesimal reveal rate.

$$w(t)\ell_{\theta} = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \ell_{\theta}(z_t, x_0)$$

The remaining loss uses the correct clean-token probability

Define the error only at currently masked positions.

$$\ell_{\theta}(z_t, x_0) = \sum_{i: z_t^i = m} -\log d_{\theta}^i(x_0^i \mid z_t, t)$$

Weight by the infinitesimal reveal rate.

$$w(t)\ell_{\theta} = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \ell_{\theta}(z_t, x_0)$$

The weighted error $w(t)\ell_{\theta}$ equals negative survival derivative $-\dot{\alpha}_t$ divided by masking probability one minus α_t , multiplied by masked-token loss $\ell_{\theta}(z_t, x_0)$ for noisy sequence z_t and clean target x_0 .

The remaining loss uses the correct clean-token probability

Define the error only at currently masked positions.

$$\ell_{\theta}(z_t, x_0) = \sum_{i: z_t^i = m} -\log d_{\theta}^i(x_0^i \mid z_t, t)$$

Weight by the infinitesimal reveal rate.

$$w(t)\ell_{\theta} = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \ell_{\theta}(z_t, x_0)$$

Average over sampled times, data, and corruptions.

$$\mathcal{L}_{\text{mask}} = \mathbb{E}_{t, x_0, z_t} [w(t)\ell_{\theta}(z_t, x_0)]$$

The remaining loss uses the correct clean-token probability

Define the error only at currently masked positions.

$$\ell_{\theta}(z_t, x_0) = \sum_{i: z_t^i = m} -\log d_{\theta}^i(x_0^i \mid z_t, t)$$

Weight by the infinitesimal reveal rate.

$$w(t)\ell_{\theta} = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \ell_{\theta}(z_t, x_0)$$

Average over sampled times, data, and corruptions.

$$\mathcal{L}_{\text{mask}} = \mathbb{E}_{t, x_0, z_t} [w(t)\ell_{\theta}(z_t, x_0)]$$

The training loss $\mathcal{L}_{\text{mask}}$ equals the average over time t , clean sequence x_0 , and corrupted sequence z_t of reveal-rate weight $w(t)$ multiplied by masked-token prediction loss $\ell_{\theta}(z_t, x_0)$.

The remaining loss uses the correct clean-token probability

Define the error only at currently masked positions.

$$\ell_{\theta}(z_t, x_0) = \sum_{i: z_t^i = m} -\log d_{\theta}^i(x_0^i \mid z_t, t)$$

Weight by the infinitesimal reveal rate.

$$w(t)\ell_{\theta} = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \ell_{\theta}(z_t, x_0)$$

Average over sampled times, data, and corruptions.

$$\mathcal{L}_{\text{mask}} = \mathbb{E}_{t, x_0, z_t} [w(t)\ell_{\theta}(z_t, x_0)]$$

The training loss $\mathcal{L}_{\text{mask}}$ equals the average over time t , clean sequence x_0 , and corrupted sequence z_t of reveal-rate weight $w(t)$ multiplied by masked-token prediction loss $\ell_{\theta}(z_t, x_0)$. Visible-token KL terms are zero under SUBS; exact endpoints remove the boundary losses.

Minimizing this loss increases that probability

Minimizing this loss increases that probability

$$\dot{\alpha}_t \leq 0 \quad \Longrightarrow \quad w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \geq 0$$

Minimizing this loss increases that probability

$$\dot{\alpha}_t \leq 0 \quad \implies \quad w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \geq 0$$

Because the survival probability decreases, its derivative $\dot{\alpha}_t$ is nonpositive, so the loss weight $w(t)$, equal to $-\dot{\alpha}_t$ divided by the masking probability $1 - \alpha_t$, is nonnegative.

Minimizing this loss increases that probability

$$\dot{\alpha}_t \leq 0 \quad \implies \quad w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \geq 0$$

Because the survival probability decreases, its derivative $\dot{\alpha}_t$ is nonpositive, so the loss weight $w(t)$, equal to $-\dot{\alpha}_t$ divided by the masking probability $1 - \alpha_t$, is nonnegative.

$w(t)$ controls how strongly errors at time t contribute to the loss.

Minimizing this loss increases that probability

$$\dot{\alpha}_t \leq 0 \quad \implies \quad w(t) = \frac{-\dot{\alpha}_t}{1 - \alpha_t} \geq 0$$

Because the survival probability decreases, its derivative $\dot{\alpha}_t$ is nonpositive, so the loss weight $w(t)$, equal to $-\dot{\alpha}_t$ divided by the masking probability $1 - \alpha_t$, is nonnegative.

$w(t)$ controls how strongly errors at time t contribute to the loss.

Minimizing a positively weighted cross-entropy increases the probability of the correct clean token.

A short aside on a first-order expansion